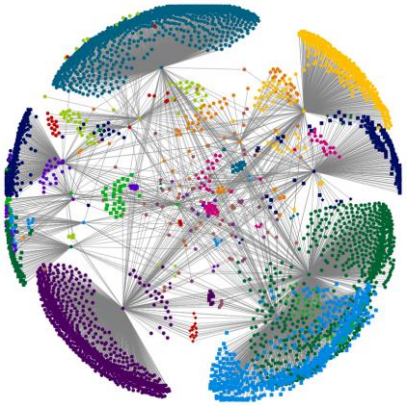
Automating Parallel Query Execution for Massive Graph Data over a Shared-nothing Cluster

Chongyang Xu*

*work done when he was an engineer intern at Alibaba Group

Massive graph datasets in real world

Social networks



Webpage networks

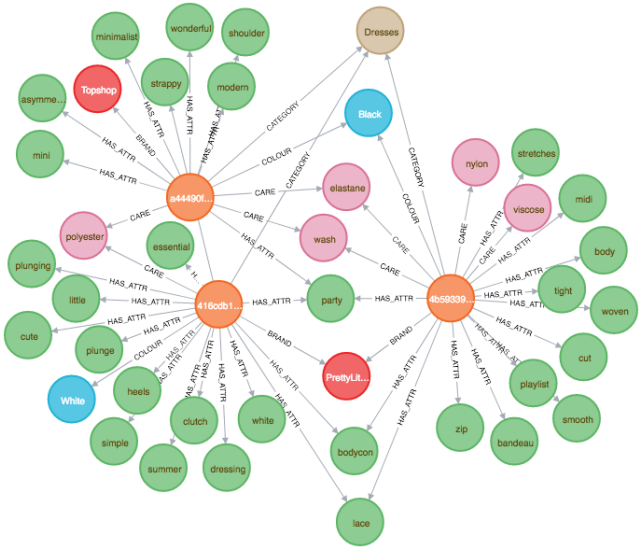
Citation networks

E-commerce knowledge graph

Supply chain graph

Phone call graph

Requiring system for big graph query



E-commerce knowledge graph

Analysts, often having no CS background, want to know “results”, e.g.

- What products did customer Dan buy?
- What products did Dan buy that was produced by vendor Leica and cost more than €2000?

Analysts don’t handle implementation detail:

- Dealing with large dataset up to TB
- Programming with a cluster of computers

Existing systems

	High abstraction level	High efficiency
Graph database	✓	✗
Graph system	✗	✓

Existing systems

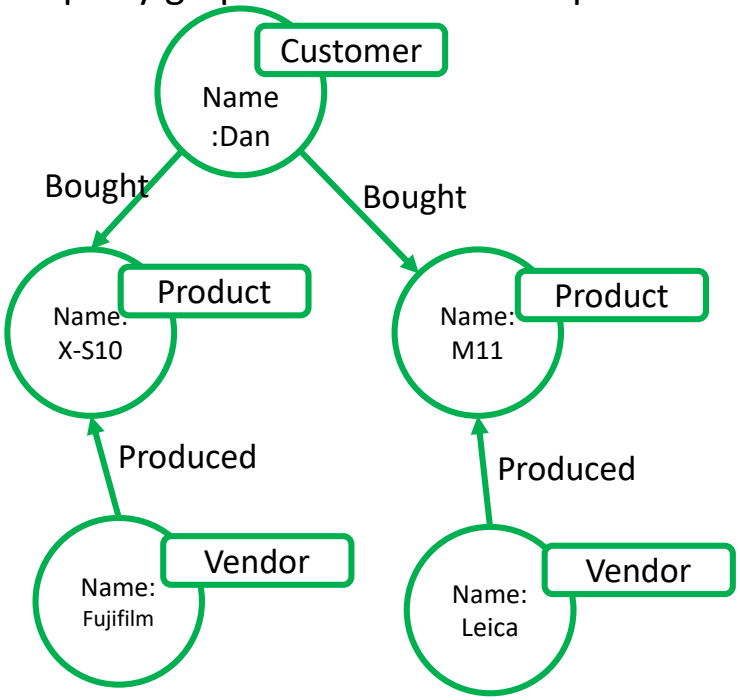
	High abstraction level	High efficiency
Graph database	✓	✗
Graph system	✗	✓

Our goal: provide both **intuitive** and **efficient** query by combining strengths of graph database and system

Graph database

Provides high level query language as interface,
Cypher is a representative one, it's created by neo4j

Property graph data model example



Cypher syntax example

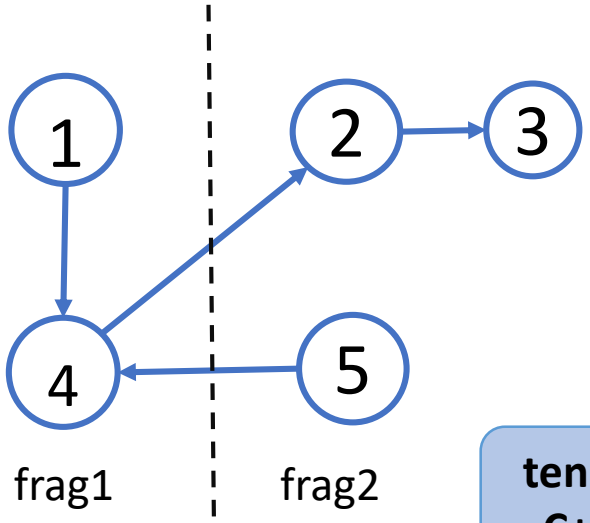
```
MATCH  (:Customer{name:"Dan"})
       -[:BOUGHT]->
       (what)
RETURN what
```

```
MATCH  (:Customer{name:"Dan"})
       -[:BOUGHT]->
       (what)
       <-[:PRODUCED]-
       (:Vendor{name:"Leica"})
WHERE  what.price > €2000
RETURN what
```

Intuitive interface, but lacking high efficiency

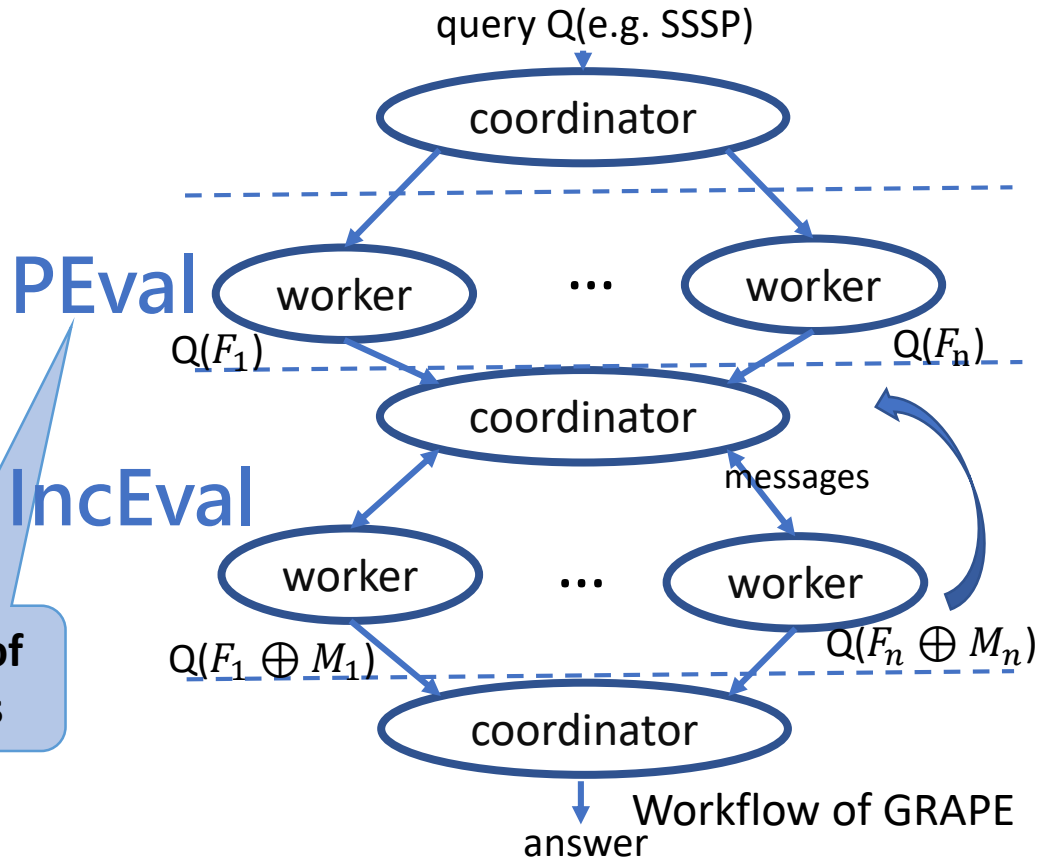
Graph system

GRAPE(sigmod 17) is a graph engine with state-of-the-art performance



Fragmented data

tens lines of C++ codes

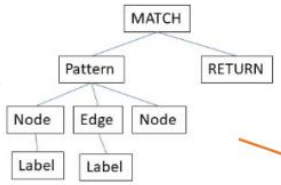


Workflow of GRAPE

High performance via parallel, but high coding efforts for non-CS analysts

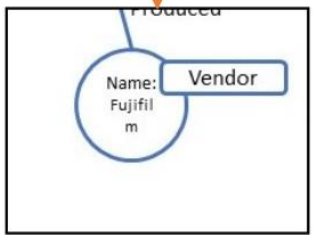
Combining Cypher and GRAPE

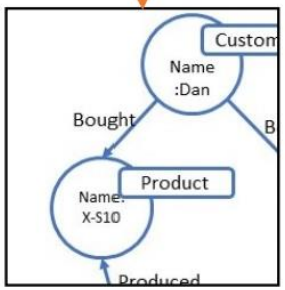
```
Match (:Customer{name:"Dan"})
      -[:BOUGHT]->
      (what)
RETURN what
```

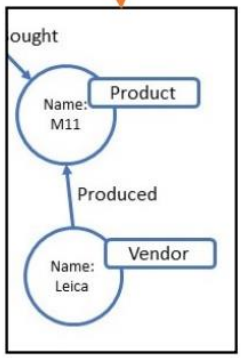


Gap

Execute program in parallel







Challenge 1

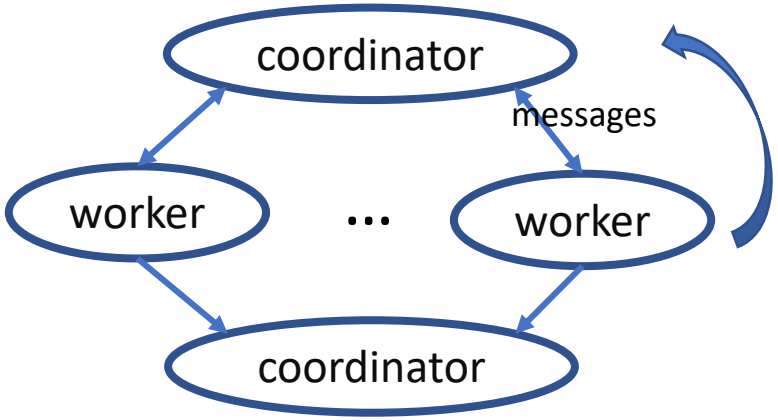
The interface we want

```
MATCH (c:Customer) WHERE c.age > 24
RETURN c
```

```
MATCH (c:Customer), (d:Customer)
WHERE c.age = d.age
RETURN c, d
```

```
MATCH (c:Customer)
WHERE c.age =24 OR (c)-[:BOUGHT]->(what)
RETURN c
```

The execution engine we have



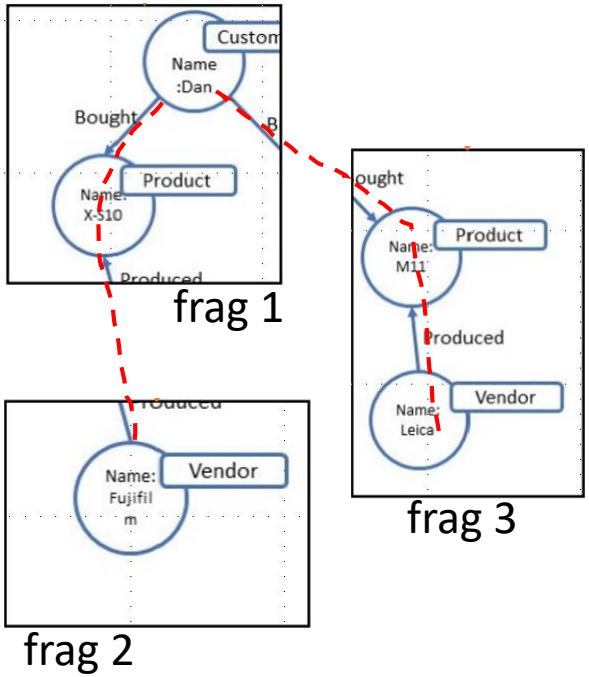
It's unclear how to execute any possible query in parallel

Challenge 2

2-hop query example

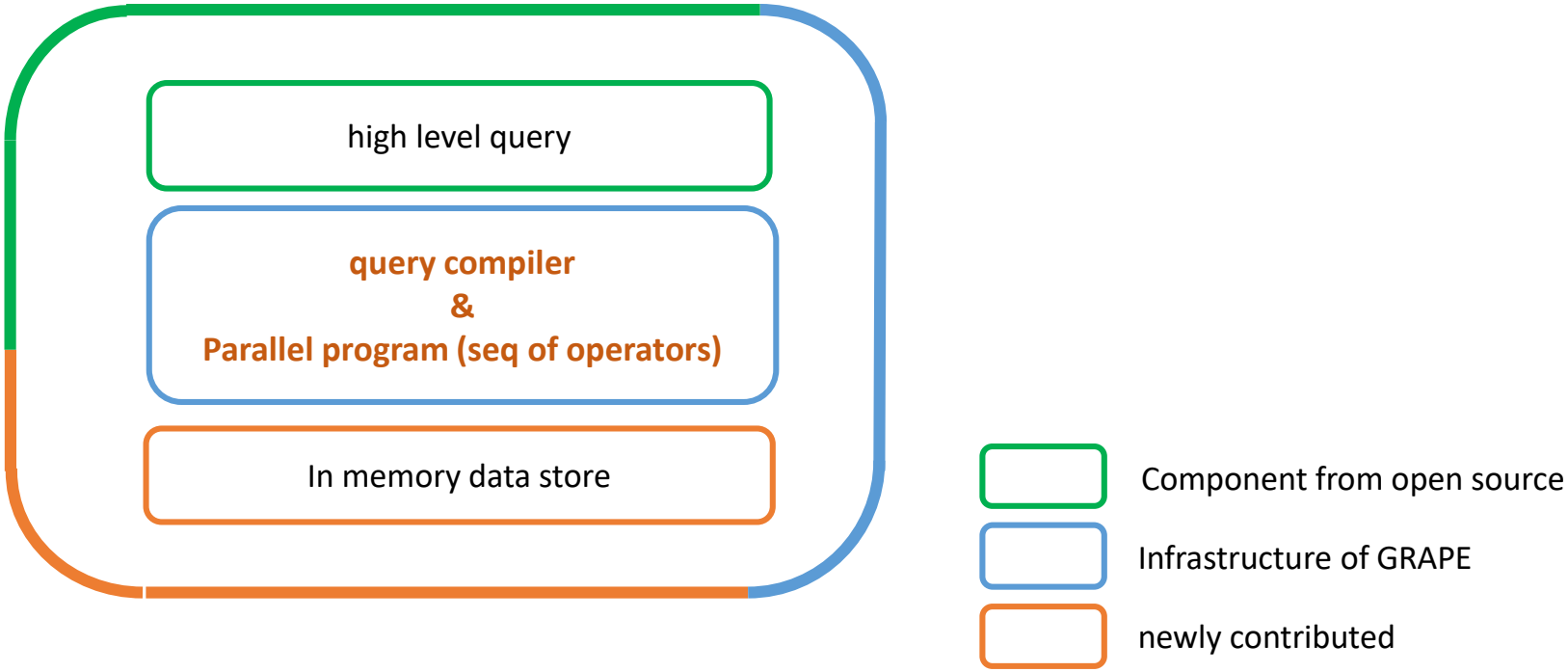
```
MATCH   (:Customer{name:"Dan"})
        -[:BOUGHT]->
        (what)
        <-[:PRODUCED]-
        (:Vendor{name:"Leica"})
WHERE   what.price > €2000
RETURN  what
```

fragments across workers



It's hard to query a path across fragment boundary

System Overview



Contributions

- new operator set for various queries
  - such as NodeScan, Neighbor, FragFilter
- new Cypher compiler backend
  - automatically convert a AST into replicated sequence of operators which execute in BSP manner
- A system to automatically execute query in parallel

- Thanks